GravityLens

> **Your Cloud Infrastructure Has a Story. Now You Can Watch It.**

GravityLens is an enterprise-grade **Cloud Infrastructure Intelligence Platform** that visualizes, tracks, and replays your AWS infrastructure evolution over time. Think of it as **GitHub for Cloud Infrastructure** — every change is captured, versioned, and visualizable.

Table of Contents

What is GravityLens?

GravityLens is a platform that answers these critical questions about your AWS account:

1. **What infrastructure do I have?** (discovery)
2. **How is it connected?** (relationships)
3. **How has it changed?** (versioning)
4. **What changed between two points in time?** (diff)
5. **Can I see those changes as a movie?** (replay)

Core Value Proposition

Instead of manually piecing together your AWS architecture from the AWS Console, GravityLens:

- **Scans automatically** — connects your AWS account once, then continuously discovers all resources
- **Visualizes intelligently** — shows your infrastructure as an interactive graph (not a table or list)

- **Tracks history** — creates immutable snapshots after each scan, preserving infrastructure versions
- **Reveals relationships** — shows which services talk to each other (EC2 → RDS, Lambda → S3, API Gateway → SQS → Lambda)
- **Enables replay** — watches infrastructure evolution as an animation, not a static snapshot

Who It's For

- **DevOps engineers** — understand what they've built without digging through 6 AWS consoles
- **Site reliability engineers (SREs)** — quickly debug "what changed?" when production breaks
- **Cloud architects** — validate that infrastructure matches design intent
- **Security teams** — audit relationships and spot unusual configurations
- **Cost managers** — see which services are connected and calculate cross-AZ data transfer costs
- **New team members** — onboard faster by watching infrastructure instead of reading docs

---

The Problem

AWS is powerful. AWS is also complicated.

When you have 50+ resources across multiple regions, VPCs, and availability zones, the following happens:

Problem 1: AWS Architecture is Invisible

The Symptom

You ask your team: *"What services do we have in production?"*

Answers you get:

- "Uh... we have some Lambda functions. And RDS. And S3."
- "Let me check... *spends 20 minutes in AWS Console tabs*"
- "I think we have 3 VPCs but I'm not sure which ones are active"

Why It Happens

The AWS Console is **designed for managing individual resources**, not visualizing an entire architecture:

- **Multi-console scatter** — EC2 resources are here, RDS here, Lambda there, VPC there. It's like having your infrastructure scattered across 10 different filing cabinets.
- **List-based interface** — the console shows resources as tables and lists. An EC2 instance is

a row. You don't see it's inside a subnet, which is inside a VPC, which is behind an API Gateway.

- **No holistic view** — even AWS's own architecture diagram tool (AWS Diagram Tool) requires manual drawing. There is no "show me everything" button that works.
- **Region switching fatigue** — your Lambda is in `ap-south-1`, but your RDS is in `us-east-1`. You constantly switch regions to see different pieces of the puzzle.

The Business Impact

- **Onboarding takes weeks** — new engineers spend days just understanding what the architecture looks like
- **Risky changes** — teams make changes without fully understanding relationships, causing cascading failures
- **Hidden costs** — nobody knows why data transfer bills are high because relationships are invisible
- **Audit failures** — security teams can't quickly verify that infrastructure matches policy

---

Problem 2: Change Tracking is Manual

The Symptom

At 3 AM, your production API stops responding.

Your team asks: *"What changed in the last 2 hours?"*

What happens next:

- You check CloudTrail (if it's enabled) and see 500+ API calls
- You manually parse which ones matter
- You try to understand the cascading effect
- By 4 AM, you *might* find the culprit

Why It Happens

AWS does NOT maintain a version history of your infrastructure:

- **No infrastructure commits** — when you add a Lambda to a VPC, AWS doesn't record "version 47 of your infrastructure." It just updates the resource.
- **CloudTrail is audit logs, not architecture history** — CloudTrail shows API calls, not infrastructure states. You know someone called `create_security_group`, but you don't know what the entire security group configuration was, or how it affected your architecture.
- **Manual documentation** — teams use Confluence/Notion to document architecture, but it's constantly out of sync with reality.

- **No "before and after"** — if you want to compare your infrastructure from yesterday to today, you're stuck manually listing resources twice.

The Business Impact

- **MTTR skyrockets** — debugging incidents takes 3x longer because you're hunting through logs
- **Risky rollbacks** — you can't easily revert a bad infrastructure change because you don't remember what it was before
- **Compliance headaches** — auditors ask "what changed on this date?" and you have no answer
- **Knowledge loss** — when engineers leave, they take the architecture knowledge with them

---

Problem 3: Understanding Relationships is Hard

The Symptom

You have a Lambda that processes images. You need to know:

- Which VPC is it in?
- Which subnets can it access?
- Which S3 buckets does it write to?
- Which RDS database does it query?
- What API Gateway route invokes it?
- Which SQS queue triggers it?

Getting this information requires:

1. Open Lambda console → find the function
2. Click "VPC configuration" → see the subnet IDs
3. Open VPC console → find those subnets
4. Open EC2 console → find security groups attached
5. Open S3 console → check bucket policies
6. Open RDS console → find the database
7. Open API Gateway console → find the integration
8. Open SQS console → check event source mappings

Total time: 15-20 minutes. And you're still not 100% sure you got everything.

Why It Happens

AWS doesn't visualize relationships:

- **Relationships live in multiple places** — a Lambda's S3 access is defined in its IAM role. Its SQS trigger is a separate config. Its VPC assignment is different. There's no single "here are all connections" view.
- **Implicit vs explicit relationships** — some relationships are explicit (a Lambda is "in" a VPC). Others are implicit (a Lambda "can call" an S3 bucket, based on its IAM role). AWS doesn't distinguish.
- **No graph visualization** — AWS has no built-in tool to say "show me all resources this Lambda talks to" or "show me all ways data flows through my architecture."

The Business Impact

- **Slow incident response** — when a service fails, understanding what depends on it takes too long
- **Missed security issues** — you don't see that a Lambda has overly broad S3 permissions because you're not looking at its relationships holistically
- **Over-engineered solutions** — without understanding what's connected, teams add redundancy they don't need
- **Bottleneck discovery** — you don't know which services are "hub" services that everything depends on

---

Problem 4: Debugging Production Issues is Slow

The Symptom

Your production API is returning 500 errors.

Your checklist:

- ☐ Is the API Gateway working?
- ☐ Is the Lambda being invoked?
- ☐ Is the Lambda throwing errors?
- ☐ Can the Lambda access the RDS database?
- ☐ Can the Lambda access the S3 bucket?
- ☐ Are there network issues between Lambda and RDS?
- ☐ Is RDS running out of connections?

Current debugging workflow:

1. Check CloudWatch logs for the Lambda (if you know which one is invoked)
2. Check RDS monitoring for CPU/connections
3. Check security group rules to see if traffic is allowed
4. Check IAM policies to see if permissions exist

5. Check VPC Flow Logs to see if traffic is flowing
6. Cross-reference everything manually

Total debugging time: 30-60 minutes. Because you're treating each component as isolated, not as a connected system.

Why It Happens

AWS provides **isolated monitoring, not systemic monitoring**:

- **Service-specific dashboards** — Lambda has CloudWatch, RDS has its own monitoring, API Gateway has its metrics. None of them show relationships.
- **No request tracing by default** — you don't see a request's full journey (API Gateway → Lambda → RDS) without explicitly enabling X-Ray.
- **Manual correlation** — you have to manually correlate logs from 3+ services to understand what happened.

The Business Impact

- **Long incident resolution times** — 1-hour issues become 3-hour incidents
- **Customer impact** — while you're debugging, customers are experiencing outages
- **Toil** — operations teams spend 40% of their time on this "what went wrong?" investigation
- **Burnout** — on-call engineers burn out faster because debugging is exhausting

---

Current Solutions and Their Limits

Before building GravityLens, we researched existing solutions. Here's what we found:

AWS-Native Tools

1. AWS Console (Free, comes with AWS)

- Shows individual resources
- No holistic architecture view
- No relationship visualization
- No infrastructure versioning
- No replay/history
- **Verdict:** Great for managing resources, terrible for understanding architecture

2. AWS CloudTrail (Free to use, paid for analysis)

- Records all API calls
- Audit logs, not infrastructure history
- Can't answer "what did my infrastructure look like yesterday?"

- No visualization
- **Verdict:** Essential for compliance, useless for architecture understanding

3. AWS Config (Paid, ~$2-5/month per resource)

- Tracks resource configuration changes
- Relationships API exists
- No visualization
- Complex UI, steep learning curve
- Focuses on compliance, not architecture understanding
- **Verdict:** Good for compliance audits, not good for daily architecture work

4. AWS Application Discovery Service (Closed to new customers as of Nov 2025)

- Discovers on-premises infrastructure
- Doesn't solve AWS-to-AWS relationship discovery
- Was designed for migration planning, not ongoing management
- **NO LONGER AVAILABLE for new customers**
- **Verdict:** Not applicable anymore

Third-Party Solutions

1. CloudViz.io (2018, acquired)

- Visualizes AWS architecture
- No infrastructure versioning
- No replay/animation
- Expensive ($$$)
- No longer actively developed

2. Brainboard.co (2015, still active)

- Full IaC platform with visualization
- Can version infrastructure
- Requires manual IaC management
- Not for existing architecture (only new projects)
- Very expensive
- **Verdict:** For new projects, not for existing AWS accounts

3. Lucidchart / Draw.io

- Great for drawing diagrams
- Manual updates required
- No real-time discovery
- No versioning
- **Verdict:** Good for documentation, not for actual architecture tracking

4. Hava.io (2015, still active)

- Auto-discovers AWS architecture
- Generates diagrams
- No versioning or replay
- Expensive ($$$)
- Diagrams are static, not interactive
- **Verdict:** Best existing solution, but still missing versioning and replay

5. Datadog / New Relic / Splunk

- Excellent monitoring and tracing
- Designed for observability, not architecture visualization
- Expensive for architecture-only use cases
- No infrastructure versioning
- **Verdict:** Complementary to architecture tools, not a replacement

---

Why GravityLens is Different

GravityLens solves all four problems with a **single, elegant approach:**

1. Automatic Discovery

```
1. Connect AWS account (paste IAM Role ARN)
2. GravityLens scans all 8 services automatically
3. You see your architecture in 60 seconds
```

No manual drawing. No IaC required. No documentation to maintain.

2. Relationship-First Design

Instead of showing resources as a list, GravityLens shows them as a **graph**:

```
API Gateway
    ↓
Lambda (process-orders)
    ├→ RDS (orders-db)
    ├→ S3 (order-receipts)
```

```
    └→ SQS (notification-queue)
         ↓
    Lambda (send-email)
```

Every edge means something. Not just "these resources exist," but "these resources are connected."

3. Infrastructure Versioning (Like Git, But for Cloud)

Each time you scan:

- We capture a **complete snapshot** of your infrastructure
- We assign it a **version number** (v1, v2, v3...)
- We make it **immutable** (can't change v1, it's permanent)
- We calculate **what changed** between versions automatically

```
Version 1 (2026-06-01 08:00)
├── 3 VPCs, 8 Subnets, 5 Lambda functions, 2 RDS databases
└── Total: 32 resources

Version 2 (2026-06-01 14:00)
├── 3 VPCs, 8 Subnets, 6 Lambda functions, 2 RDS databases
├── ADDED: Lambda (process-refunds)
├── MODIFIED: Lambda (process-orders) - memory increased 512→1024MB
└── Total: 33 resources

Version 3 (2026-06-02 09:00)
├── 3 VPCs, 8 Subnets, 5 Lambda functions, 2 RDS databases
├── REMOVED: Lambda (process-refunds) - no longer needed
└── Total: 32 resources
```

4. Replay Animation

Once you have versions, you can **watch your infrastructure evolve**:

```
Play version-1 → version-3

Timeline: ═══════●═══════●═══════●
          v1      v2      v3

Animation sequence:
1. Remove (fade out): Lambda (process-refunds) disappears with a dissolve
2. Modify (highlight): Lambda (process-orders) box pulses yellow, memory label
changes
3. Add (appear): (nothing new added in this direction)
```

This is the **"A-ha!" moment** — seeing infrastructure change over time as a movie, not a timeline of events.

5. Fast Incident Response

When production breaks:

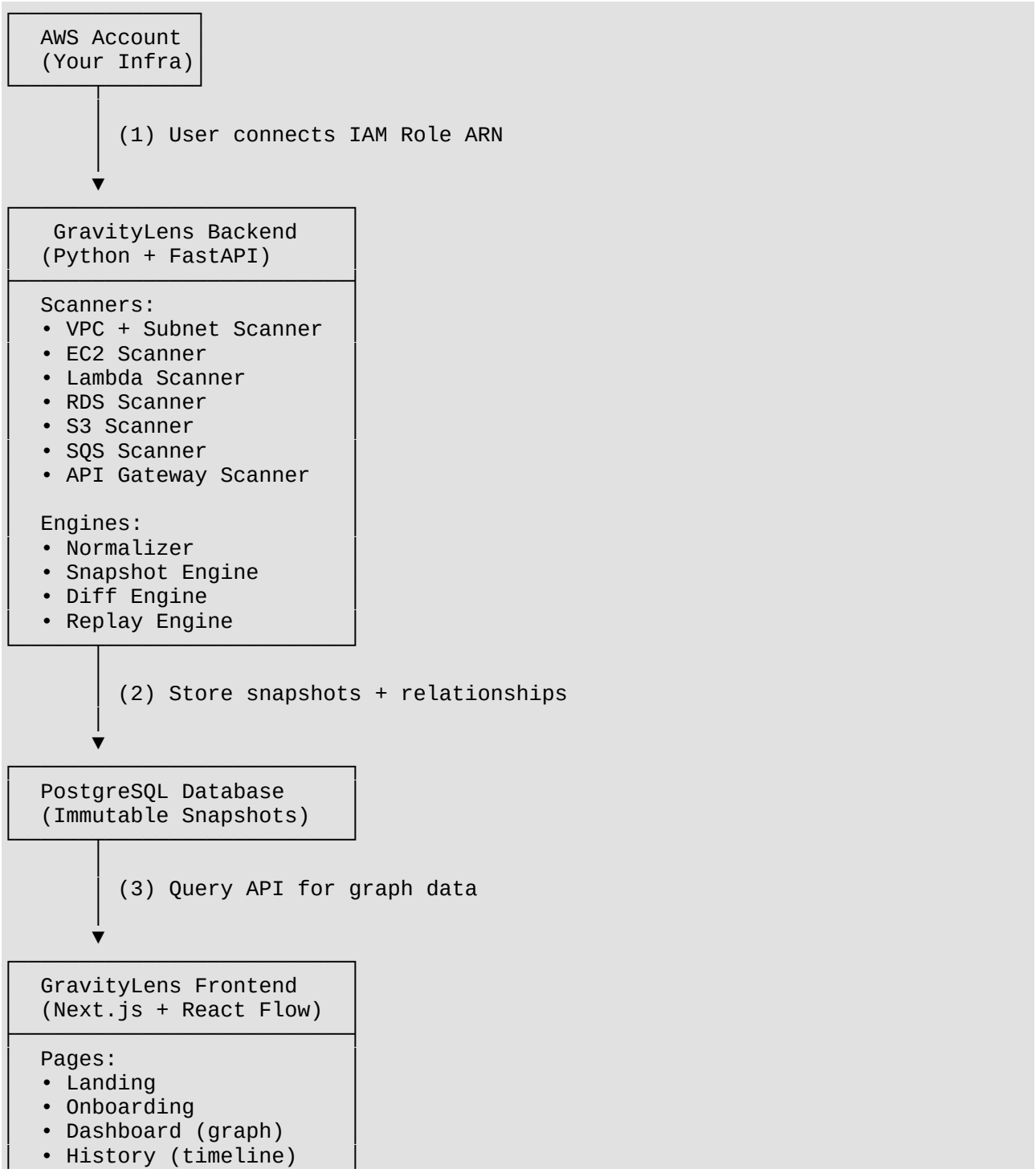
- Open dashboard → see your current architecture at a glance
- Click on the failing service → see what it's connected to

- Jump to "History" tab → see what changed in the last 2 hours
- Compare versions → highlight exactly what changed
- Watch replay → see the change happen as an animation

Result: 3x faster incident resolution.

How It Works

Architecture Flow



```
  • Diff Viewer
  • Replay (animation)
```

The User Journey

Minute 0-1: Connect

```
User opens GravityLens
→ Clicks "Connect AWS"
→ Pastes IAM Role ARN
→ Clicks "Verify"
```

Minute 1-2: Scan

```
Backend starts scan:
├─ Discover regions
├─ Scan VPCs and Subnets
├─ Scan EC2, Lambda, RDS, S3, SQS, API Gateway
└─ Build relationships
```

Minute 2-3: See

```
Frontend displays:
├─ Overview cards (8 EC2, 3 VPC, 5 Lambda, 2 RDS)
├─ Interactive graph (nodes + edges)
└─ Resource detail panel
```

Minute 3+: Explore

```
User can now:
├─ Click nodes to see details
├─ View historical versions
├─ Compare two versions
└─ Watch replay animation
```

The GravityLens Experience

What You See

Dashboard (Main View)

```
┌──────────────────────────────────────────────────┐
│  GravityLens                    🔍 Dashboard     │
├──────────────────────────────────────────────────┤
│                                                  │
│  Overview Cards:                                 │
│  ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐            │
│  │EC2   │ │VPC   │ │RDS   │ │Lambda│            │
│  │  8   │ │  3   │ │  2   │ │  5   │            │
│  └──────┘ └──────┘ └──────┘ └──────┘            │
│                                                  │
│  Infrastructure Graph:                           │
│  ┌──────────────────────────────────────┐        │
│  │  [VPC-1]                             │        │
│  │    ├─ [Subnet-1]                     │        │
│  │    │   ├─ [EC2-web]  ←─┐             │        │
│  │    │   └─ [Lambda]     │             │        │
```

```
   │         │                          ├→ [RDS]      │   │
   │         └─ [Subnet-2]              │             │   │
   │               └─ [Lambda]          │             │   │
   │                                    │             │   │
   │    [S3-bucket] ←───────────────────┘             │   │
   │                                                  │   │
   │    [API-Gateway] ──→ [SQS] ──→ [Lambda]           │
   └──────────────────────────────────────┘           │
                                                          │
  Click any node → see details                            │
                                                          │
└─────────────────────────────────────────────────────────┘
```

History (Timeline View)

```
┌─────────────────────────────────────────────────────────┐
│  GravityLens                         🕐 Timeline          │
├─────────────────────────────────────────────────────────┤
│                                                          │
│  ● Version 3  (Latest)                                   │
│    2026-06-15 14:30                                      │
│    8 EC2 · 3 VPC · 5 Lambda · 2 RDS                      │
│    +1 Added  -0 Removed  ~1 Modified                     │
│    [View] [Compare] [Replay]                             │
│    │                                                     │
│    │                                                     │
│  ● Version 2                                             │
│    2026-06-15 08:00                                      │
│    7 EC2 · 3 VPC · 5 Lambda · 2 RDS                      │
│    +1 Added  -0 Removed  ~0 Modified                     │
│    [View] [Compare] [Replay]                             │
│    │                                                     │
│    │                                                     │
│  ● Version 1  (First Scan)                               │
│    2026-06-14 15:30                                      │
│    6 EC2 · 3 VPC · 4 Lambda · 2 RDS                      │
│    [View]                                                │
│                                                          │
└─────────────────────────────────────────────────────────┘
```

Diff Viewer (What Changed)

```
┌─────────────────────────────────────────────────────────┐
│  GravityLens                    Compare v2 → v3           │
├─────────────────────────────────────────────────────────┤
│                                                          │
│  +1 Added  -0 Removed  ~1 Modified                       │
│                                                          │
│  ADDED (Green)                                           │
│   ┌──────────────────────────────────────┐              │
│   │ + EC2 Instance (web-server-2)        │              │
│   │   Type: t3.medium                    │              │
│   │   Region: ap-south-1                 │              │
│   └──────────────────────────────────────┘              │
│                                                          │
│  MODIFIED (Yellow)                                       │
│   ┌──────────────────────────────────────┐              │
│   │ ~ Lambda (process-orders)            │              │
│   │   Memory: 512 MB → 1024 MB           │              │
│   │   Timeout: 30s → 60s                 │              │
│   └──────────────────────────────────────┘              │
│                                                          │
```

```
  [▶ Watch Replay Animation]
```

Replay (Watch It Happen)

```
When you click "Watch Replay":

Timeline: ─●──────────────●
          v2             v3

Play button: ▶

1. Nodes fade in/out with smooth animations
2. Labels change with color highlights (yellow for modified)
3. Edges appear and disappear smoothly
4. You see your infrastructure "evolve" in 10 seconds

This is the magic moment:
You WATCH your infrastructure change instead of reading a list of changes.
```

Why This Matters

For DevOps Teams

Before GravityLens:

- "What's our production architecture?" → *spends 30 minutes exploring AWS Console*
- New engineer onboarding → *takes 3 days to understand infra*
- Production incident → *takes 1 hour to understand what broke*

After GravityLens:

- "What's our production architecture?" → *opens dashboard, sees it in 30 seconds*
- New engineer onboarding → *watches replay, understands in 5 minutes*
- Production incident → *opens dashboard + history, understands in 5 minutes*

For Architects

Before GravityLens:

- Can't validate that deployed infra matches design
- No way to communicate architecture to stakeholders
- Architecture documentation is always out of sync

After GravityLens:

- See exactly what was deployed vs what was designed
- Share interactive graph with stakeholders (not static diagrams)
- Documentation stays in sync automatically

For Security Teams

Before GravityLens:

- Manual audits to understand relationships
- Can't see if services have overly broad permissions
- Hidden dependencies create audit failures

After GravityLens:

- Relationships are immediately visible
- Can spot misconfigured services at a glance
- Audit trails are automatic (every version is recorded)

For Cost Management

Before GravityLens:

- Can't explain why data transfer costs are high
- Cross-AZ traffic is invisible
- No way to optimize without understanding architecture

After GravityLens:

- Every relationship shows transfer cost
- Cross-AZ traffic is visualized
- Optimization opportunities become obvious

The Core Philosophy

GravityLens is built on three principles:

1. Infrastructure as a Story

Your AWS account is not a static thing — it's constantly evolving. We treat it like a story with chapters (versions), scenes (resources), and plot points (relationships). You should be able to tell that story.

2. Show, Don't Tell

Instead of showing you lists and tables, we *show* you your architecture as an interactive graph. You see relationships, not configurations. You see changes as animations, not as text diffs.

3. One Source of Truth

Your infrastructure source of truth should not be:

- A Notion document (constantly out of sync)

- A Terraform file (not everyone uses Terraform)
- A CloudFormation template (not everyone uses CF)
- Your memory (unreliable)

It should be: **The actual AWS account itself.**

GravityLens uses the AWS account as the single source of truth, discovers everything automatically, and lets you explore it interactively.

What's Coming Next

GravityLens is being built in phases:

Phase 1: MVP (Now)

- 8 AWS services (VPC, Subnet, EC2, Lambda, RDS, S3, SQS, API Gateway)
- Automatic discovery
- Interactive graph visualization
- Relationship discovery (structural + application wiring)

Phase 2: Enhanced Discovery

- Runtime relationship discovery (X-Ray)
- Network traffic analysis (VPC Flow Logs)
- Cost calculation per edge

Phase 3: Intelligence Layer

- Anomaly detection
- Security recommendations
- Performance insights
- Cost optimization suggestions

Phase 4: Enterprise

- Multi-account support
- Team collaboration
- Custom integrations
- Advanced analytics

Conclusion

AWS is powerful, but AWS's own tools don't help you see the full picture. GravityLens fixes that.

Instead of treating your infrastructure as scattered resources across multiple consoles, GravityLens treats it as a **connected system** that you can see, understand, and explore.

The goal is simple:

> When someone asks "what's our production architecture?", the answer should take 30 seconds, not 30 minutes.

That's what GravityLens delivers.

Ready to understand your infrastructure? Get started at gravitylens.dev